

BÀI 5: KIỂM THỬ BẢNG QUYẾT ĐỊNH

Mục tiêu:

- **Về kiến thức:** cung cấp cho sinh viên kiến thức tổng quan về kiểm thử chức năng, kiến thức về kỹ thuật kiểm thử lớp bảng quyết định: cấu trúc bảng quyết định, cách lập bảng quyết định, các bước thiết kế testcase
- **Về kỹ năng:** giúp sinh viên áp dụng kỹ thuật kiểm thử bảng quyết định thiết kế được hệ thống testcase cho bài toán cụ thể

5.1 Tổng quan

Kiểm thử hàm (functional testing) là các hoạt động kiểm tra chương trình dựa trên tài liệu mô tả chức năng, yêu cầu phần mềm, hay còn gọi là *đặc tả chức năng* (functional specification). Đặc tả chức năng là tài liệu mô tả yêu cầu, hành vi của chương trình mong muốn. Tài liệu này là cơ sở để chúng ta tiến hành xây dựng các ca kiểm thử, thuật ngữ chuyên môn gọi là *thiết kế kiểm thử* (test design) và người thực hiện việc này là người thiết kế kiểm thử. Thiết kế kiểm thử là hoạt động chính trong kiểm thử hàm.

Cần nhấn mạnh rằng kiểm thử hàm là việc thiết kế các ca kiểm thử hàm của chương trình chỉ dựa trên đặc tả của chương trình mà không dựa trên việc phân tích mã nguồn của chương trình. Kiểm thử hàm còn được gọi với tên khác chính xác hơn là kiểm thử dựa trên đặc tả, hay kiểm thử hộp đen, hay kiểm thử chức năng. Còn việc thiết kế kiểm thử có dựa vào mã nguồn của chương trình được gọi là kiểm thử cấu trúc, hay kiểm thử hộp trắng.

Kiểm thử hàm là một kỹ thuật cơ bản giúp phát hiện nhiều lỗi mà kiểm thử hộp trắng không phát hiện được. Ví dụ dễ thấy nhất là các lỗi do cài đặt (implementation) không đầy đủ so với đặc tả. Nếu phần mềm thiếu hẳn một chức năng thì chúng ta dễ nhận thấy, nhưng nếu phần mềm thiếu một trường hợp của một chức năng thì việc này không dễ phát hiện nếu không kiểm tra kỹ chương trình so với đặc tả.

Tài liệu đặc tả dùng để thiết kế các ca kiểm thử có thể đơn giản là mô tả về chức năng của chương trình viết bằng ngôn ngữ của người sử dụng, nhưng thường là cả các tài liệu phân tích, thiết kế ở các mức độ chi tiết hơn với các hình vẽ, bảng

biểu, sơ đồ làm rõ hành vi của chương trình vì chúng mô tả sát hơn các tính chất hàm của chương trình mà chúng ta sẽ dựa vào đó để thiết kế kiểm thử. Chúng ta gọi chung các tài liệu này là đặc tả chức năng.

Dễ nhận thấy do chỉ dựa trên đặc tả chức năng, kiểm thử hàm có ưu điểm là có thể thực hiện được sớm, trước khi cài đặt chương trình. Thời điểm tốt để chúng ta bắt đầu thiết kế kiểm thử là khi tài liệu đặc tả đã ổn định, nhưng cũng có nhiều qui trình phần mềm hiện đại đang đẩy việc này lên sớm hơn, ngay trong giai đoạn xác định yêu cầu đã lồng ghép xác định các ca kiểm thử. Các ca kiểm thử này thường được mô tả dưới dạng các ví dụ, vừa có tác dụng giải thích, làm rõ tài liệu đặc tả, vừa có tác dụng sử dụng luôn làm các kịch bản kiểm thử, các ca kiểm thử.

Thông thường kết quả của việc thiết kế kiểm thử hàm sẽ đưa ra *đặc tả ca kiểm thử*. Tương tự như khái niệm đặc tả chương trình là mô tả chương trình mong muốn, đặc tả kiểm thử mô tả các ca kiểm thử cần được tạo ra. Từ một đặc tả ca kiểm thử chúng ta có thể tạo ra nhiều ca kiểm thử cụ thể để tiến hành kiểm tra với chương trình. Ví dụ với một hàm giải phương trình bậc hai với đầu vào là ba hệ số là a, b, c , đặc tả ca kiểm thử cho nghiệm kép là với mọi a, b, c thỏa mãn $b^2 = 4ac$. Từ đặc tả này ta có thể tạo ra các ca kiểm thử cụ thể, ví dụ $a = 1, b = 2, c = 1$, và kết quả mong đợi là nghiệm kép -1 .

Bản chất của các đặc tả ca kiểm thử là mỗi đặc tả ca kiểm thử sẽ xác định một lớp hành vi của chương trình. Do đó việc thiết kế kiểm thử hàm là việc phân tách đặc tả các hành vi có thể của chương trình thành một số lớp mà mỗi lớp này sẽ có hành vi nhất quán hay tương tự. Với ví dụ hàm giải phương trình bậc hai trên chúng ta có thể chia đặc tả thành hành vi phương trình cho nghiệm kép, hành vi phương trình cho kết quả vô nghiệm, v.v.

Việc phân tích đặc tả sẽ giúp chúng ta xác định và phân tách được các lớp này. Bên cạnh đó việc phân tích cũng thường giúp chúng ta xác định những điểm không rõ ràng, không đầy đủ của đặc tả. Đây là quá trình phân tích tổng hợp nhiều nguồn thông tin, từ đặc tả, từ kiến thức, kinh nghiệm của người thiết kế kiểm thử, do đó nó mang tính chủ công và cũng dễ mắc lỗi. Nhiều chuyên gia về thiết kế kiểm thử hàm nhiều khi vẫn bỏ sót một số ca kiểm thử quan trọng. Để giảm thiểu các sai sót này, chúng ta cần có phương pháp hệ thống giúp chia quá trình thiết kế này thành các

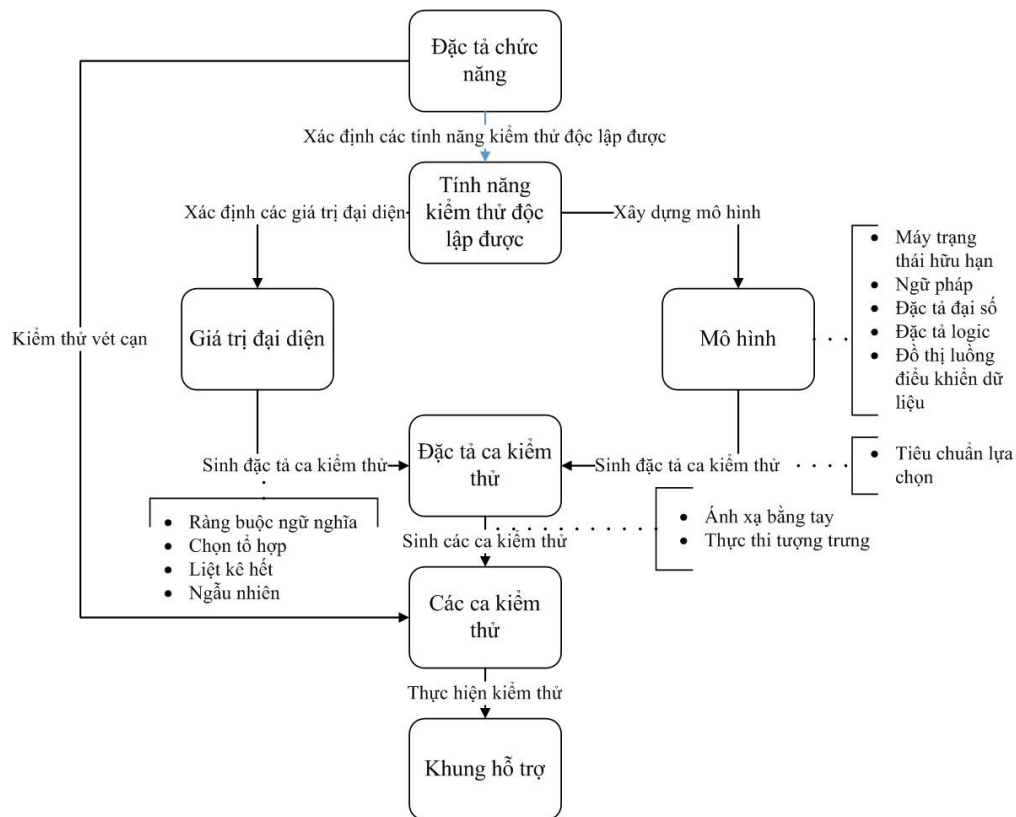
bước cơ bản để thực hiện. Từ đó một số bước sẽ được tự động hóa để giảm công sức, giảm sai sót, và tăng được chất lượng công việc. Thực tế việc tự động hoàn toàn chưa thực hiện được và một số bước cơ bản vẫn phải thực hiện thủ công. Do đó kỹ năng và kinh nghiệm của người thiết kế kiểm thử đóng vai trò quyết định đến chất lượng của bộ kiểm thử.

Để hiểu rõ hơn các công việc trong thiết kế kiểm thử hàm, chúng ta giả sử đang thiết kế kiểm thử cho chương trình P và giả sử $Y = P(X)$ trong đó X là vec-tơ của n biến đầu vào và Y là vec-tơ của m biến đầu ra. Dựa trên hiểu biết của chúng ta về đặc tả của chương trình P , với một vec-tơ đầu vào $X = (a_1, \dots, a_n)$ với a_i là các giá trị cụ thể thì chúng ta tính được một vec-tơ kết quả cụ thể $Y = (b_1, \dots, b_m)$. Kết quả Y này là được gọi là *kết quả mong đợi*. Kết quả thực của chương trình P với đầu vào X khi chạy chương trình là $P(X)$. Chúng ta cần kiểm tra kết quả mong đợi đúng bằng kết quả thực của chương trình, tức là Y và $P(x)$ là một hay không. Nếu đúng thì chương trình đã chạy đúng với X . Nếu chương trình chạy đúng với mọi giá trị X như mô tả trong đặc tả thì chúng ta kết luận chương trình P đã được cài đặt đúng so với đặc tả.

5.1.1 Phương pháp hệ thống

Việc xác định các ca kiểm thử từ đặc tả chức năng là quá trình phân tích để xác định và chia không gian đầu vào/đầu ra/hành vi của chương trình thành các miền con. Phương pháp hệ thống sẽ chia quá trình này thành các bước cơ bản, từ đó làm từng bước được dễ dàng hơn hoặc tự động hóa một số bước nếu có thể. Phương pháp hệ thống giúp chúng ta xác định bộ ca kiểm thử đã đủ chưa đồng thời tránh tạo ra nhiều ca kiểm thử trùng lặp, không cần thiết.

Hình 5.1 mô tả các bước chính của một phương pháp hệ thống. Một số bước trong qui trình này có thể phức tạp, một số bước khá đơn giản và một số bước có thể bỏ qua tùy theo miền ứng dụng và đặc tả chương trình cũng như kinh nghiệm của người thiết kế. Dựa trên qui trình tổng quát này chúng ta có thể đưa ra các qui trình cụ thể bằng việc cụ thể hóa các bước trong qui trình. Chúng ta sẽ xem xét chi tiết hơn các bước trong phương pháp này.



Hình 5.1: Các bước chính của phương pháp hệ thống cho kiểm thử hàm

Xác định các chức năng kiểm thử độc lập được: Đặc tả phần mềm trên thực tế là lớn và phức tạp ngay cả với những hệ thống nhỏ. Chúng thường được phân rã thành các đơn vị nhỏ hơn như các hệ thống con hay cụ thể hơn là các ca sử dụng (use case) hoặc các kịch bản sử dụng (user stories). Chúng ta gọi chung là các chức năng. Ví dụ trang web tìm kiếm của Google có thể có tính năng tìm kiếm, hiển thị kết quả, hiển thị quảng cáo, hay đăng ký người sử dụng mới, v.v.. Tuy nhiên ranh giới của một chức năng nhiều khi không rõ ràng. Ví dụ chức năng tìm kiếm của Google Search có chức năng tính toán một biểu thức số học nếu văn bản tìm kiếm là một biểu thức số học.

Khi thiết kế kiểm thử chúng ta tách các chức năng càng chi tiết, cụ thể càng tốt vì nó vừa giúp đơn giản việc thiết kế kiểm thử và vừa cho phép chúng ta xác định lỗi trong chương trình dễ dàng hơn. Mỗi ca kiểm thử chỉ nên kiểm tra một kịch bản cụ thể, chi tiết của một chức năng của hệ thống.

Tiếp đó với mỗi chức năng chúng ta sẽ xác định tất cả các đầu vào có ảnh hưởng đến việc thực hiện chức năng đó và các đầu ra có thể bị tác động bởi chức năng đó. Các đầu vào và đầu ra thường được mô tả tường minh trong tài liệu đặc tả. Nhưng

cũng có nhiều trường hợp các đầu vào và đầu ra này nằm ẩn trong tài liệu đặc tả mà người thiết kế phải phát hiện ra. Ví dụ như các đặc tả không hình thức mô tả chức năng đăng ký tài khoản mới thường chỉ nêu các trường dữ liệu cần khai báo nhưng không nói gì đến chi tiết kiểu dữ liệu sẽ lưu thông tin hồ sơ người sử dụng. Hay một ví dụ khác khá rắc rối với tên tiếng Việt khi chúng ta chỉ có hai trường họ và tên để nhập vào. Một là thông thường chúng ta không nói ra cụ thể chiều dài tối đa, tối thiểu, hay qui định bảng mã ký tự của các trường này. Hai là chúng ta không nói rõ phần tên đệm sẽ nằm ở phần họ hay phần tên, nên khi người sử dụng nhập dữ liệu sẽ xảy ra tình trạng mỗi người nhập một kiểu. Tên là Nguyễn Văn Hùng thì có người nhập phần họ là Nguyễn, phần tên là Văn Hùng; có người lại nhập họ là Nguyễn Văn, tên là Hùng; lại có người chỉ nhập họ là Nguyễn, tên là Hùng, vì nghĩ phần Văn là tên đệm không hỏi thì không nhập.

Xác định các lớp giá trị đại diện: Các giá trị đại diện có thể được xác định trực tiếp từ đặc tả phi hình thức, như ngôn ngữ tự nhiên. Chúng cũng có thể được xác định gián tiếp dựa trên một mô hình được xây dựng để phục vụ cho việc này. Trong cả hai trường hợp này mục tiêu đều là xác định các giá trị của mỗi đầu vào một cách độc lập, bằng việc lấy các giá trị đã liệt kê tường minh trong đặc tả, hoặc thông qua một mô hình phù hợp. Chúng ta chưa xét đến tổ hợp của các giá trị này. Lý do là chúng ta đang tách bài toán xác định các lớp giá trị của mỗi đầu vào và bài toán tổ hợp chúng để tạo thành các ca kiểm thử. Tức là chúng ta đang tách một bước phức tạp thành các bước đơn giản hơn.

Với các đặc tả không hình thức người thiết kế kiểm thử phải dựa vào việc liệt kê tường minh các lớp giá trị đại diện. Trong trường hợp này người thiết kế kiểm thử cần cẩn thận xét tất cả các trường hợp có thể và khai thác tối đa những thông tin có sẵn trong đặc tả. Tương tự, với kết quả mong đợi đầu ra, chúng ta cũng có thể xác định các lớp giá trị của miền đầu ra, cũng như các giá trị biên và các giá trị đặc biệt để có lỗi.

Ví dụ khi xem xét các thao tác trên một danh sách không rỗng chúng ta cần xét cả trường hợp danh sách rỗng (giá trị lỗi) và danh sách có một phần tử (giá trị biên chiều dài danh sách) là các trường hợp đặc biệt. Ở bước này chúng ta chỉ quan tâm đến tính chất của giá trị, chứ chưa cần cụ thể giá trị là gì. Tức là chúng ta quan tâm đến đặc tả ca kiểm thử chứ chưa phải ca kiểm thử cụ thể. Với ví dụ danh sách một

phần tử, chúng ta chưa cần quan tâm phần tử của danh sách này cụ thể là gì, chỉ biết danh sách chỉ có một phần tử.

Để liệt kê các giá trị không tường minh chúng ta cần xây dựng mô hình hoặc một phần của mô hình của đặc tả. Ví dụ chúng ta có thể dùng văn phạm để mô tả ngôn ngữ và sinh các từ của ngôn ngữ này để liệt kê các giá trị cần xác định. Mô hình này có thể có sẵn trong đặc tả nhưng thông thường người thiết kế kiểm thử phải xây dựng chúng.

Trong hai cách trên, việc liệt kê trực tiếp có vẻ đơn giản hơn và ít tốn kém hơn việc xây dựng mô hình để sinh ra các giá trị. Tuy nhiên về lâu về dài các mô hình sẽ có giá trị hơn, khi khối lượng giá trị lớn chúng ta có thể tự động hóa việc liệt kê, hay chúng ta cũng có thể điều chỉnh số lượng ca kiểm thử nhiều hay ít để cân bằng với các ràng buộc kiểm thử khác, như chi phí, thời gian. Các mô hình cũng có thể được sử dụng lại, sửa đổi dễ dàng hơn trong quá trình phát triển hệ thống. Quyết định cuối cùng nên dùng cách nào vẫn tùy thuộc vào lĩnh vực ứng dụng, kỹ năng của người thiết kế kiểm thử, và các công cụ thích hợp đang có.

Sinh các đặc tả ca kiểm thử: Đặc tả kiểm thử được tạo ra bằng việc tổ hợp các giá trị của các đầu vào của chương trình đang được kiểm thử. Bước trên chúng ta đã xác định các giá trị đại diện thì ở bước này chúng ta sinh các tổ hợp của chúng bằng tích Đề-các của các giá trị đại diện đã chọn. Nếu ở bước trước chúng ta tạo ra mô hình hình thức thì ở bước này các đặc tả ca kiểm thử sẽ là các hành vi cụ thể hoặc tổ hợp của các tham số của mô hình và một đặc tả ca kiểm thử có thể được thỏa mãn bởi nhiều đầu vào cụ thể. Trong cả hai cách này việc tạo ra tất cả các tổ hợp thường tạo ra số lượng quá lớn các ca kiểm thử.

Ví dụ một hàm có năm đầu vào, mỗi đầu vào có sáu giá trị đại diện thì tích Đề-các sẽ tạo ra $6^5 = 7,776$ đặc tả ca kiểm thử. Số lượng này là quá lớn với một chức năng, ngay cả khi được tự động hóa và chạy bằng máy tính hoàn toàn. Thông thường, rất nhiều tổ hợp là không hợp lệ hoặc không khả thi. Chúng ta sẽ học các phương pháp để giảm số tổ hợp này nhưng vẫn đảm bảo chất lượng của bộ kiểm thử.

Để giảm số ca kiểm thử, chúng ta cần loại bỏ các tổ hợp giá trị không hợp lệ bằng việc đưa thêm các ràng buộc về cách tổ hợp. Ví dụ hàm *NextDate* chúng ta cần đưa ràng buộc để tránh tạo ra các giá trị không hợp lệ như ngày 31/11/2013. Hoặc chúng

ta cũng không xét hết tất cả các tích Đề-các mà chỉ cần mỗi cặp giá trị của hai biến bất kỳ đều xuất hiện là đủ. Quá trình này có thể không phải một bước làm là xong mà có thể cần điều chỉnh và thử một số lần để có kết quả là bộ ca kiểm thử phù hợp, vừa hạn chế được các ca kiểm thử vô lý không cần thiết, vừa tiết kiệm được chi phí thực hiện kiểm thử.

Sinh ca kiểm thử và thực hiện kiểm thử: Quá trình kiểm thử hoàn tất khi chúng ta cụ thể hóa các đặc tả kiểm thử thành các ca kiểm thử cụ thể, rồi tính kết quả mong đợi của từng ca kiểm thử, và tiến hành kiểm tra với phần mềm. Việc thực hiện kiểm thử cần được tự động hóa tối đa khi có thể, vì một hệ thống phần mềm cần được kiểm thử rất nhiều lần. Ngay cả khi sản phẩm hoàn tất, trong tương lai chúng ta vẫn sẽ cần thực hiện kiểm thử nhiều lần nữa, khi phần mềm tiến hóa, thay đổi theo những yêu cầu mới, hay các lỗi được phát hiện và phần mềm cần phải sửa.

5.1.2 Lựa chọn phương pháp phù hợp

Trong các chương sau chúng ta sẽ thấy một số kỹ thuật kiểm thử hàm thích hợp cho các loại đặc tả chương trình khác nhau. Với một đặc tả có thể có một số kỹ thuật phù hợp để xây dựng các ca kiểm thử và một số kỹ thuật sẽ rất khó áp dụng hoặc kết quả sẽ không thỏa mãn. Một số kỹ thuật có thể cho kết quả như nhau với một số đặc tả. Một số kỹ thuật lại bổ sung cho nhau nên cần kết hợp chúng. Ví dụ như khi mỗi kỹ thuật lại áp dụng tốt cho những khía cạnh khác nhau của cùng một đặc tả, hoặc ở các giai đoạn khác nhau của việc sinh ca kiểm thử.

Việc lựa chọn kỹ thuật để tạo ra các ca kiểm thử phụ thuộc vào các yếu tố: tính tự nhiên của đặc tả, dạng của đặc tả, kinh nghiệm của người thiết kế kiểm thử, cấu trúc của tổ chức, công cụ sẵn có, ngân sách và các ràng buộc về chất lượng, chi phí thiết kế và cài đặt các hệ thống hỗ trợ.

Tính tự nhiên và dạng của đặc tả: Các kỹ thuật kiểm thử khai thác các đặc điểm khác nhau của đặc tả. Ví dụ, khi biên đầu vào có một số ràng buộc thì có thể kỹ thuật kiểm thử lớp tương đương sẽ phù hợp. Nếu có sự chuyển dịch trong các trạng thái của hệ thống thì phương pháp kiểm thử dựa trên máy trạng thái có thể nên áp dụng. Hay khi các đầu vào thay đổi với kích thước không xác định được thì phương pháp dùng ngữ pháp có thể phù hợp.

Kinh nghiệm của người thiết kế kiểm thử: Kinh nghiệm của người kiểm thử

hiều khi sẽ ảnh hưởng đến kỹ thuật kiểm thử nào được sử dụng. Ví dụ một người thiết kế kiểm thử giỏi về bảng quyết định sẽ thích kỹ thuật đó khi một kỹ thuật khác cũng có thể áp dụng được.

Công cụ: Một số kỹ thuật đòi hỏi sử dụng các công cụ mà đôi khi phải mua nên chi phí cũng là một yếu tố ảnh hưởng đến việc lựa chọn kỹ thuật kiểm thử. Ví dụ đặc tả sử dụng UML và có công cụ thương mại sinh ca kiểm thử dựa trên một số biểu đồ UML thì công cụ đóng vai trò quan trọng ảnh hưởng đến phương pháp thực hiện kiểm thử, nếu ngân sách cho kiểm thử cho phép. Một số đơn vị làm phần mềm chuyên nghiệp đôi khi phải xây dựng các công cụ này để phục vụ cho đơn vị, hoặc đôi khi cho từng dự án cụ thể.

Ngân sách và ràng buộc chất lượng: Ngân sách và ràng buộc chất lượng có thể dẫn đến sự lựa chọn khác nhau về phương pháp kiểm thử. Ví dụ nếu yêu cầu là nhanh, kiểm thử tự động, nhưng không cần độ tin cậy cao thì kiểm thử ngẫu nhiên là phù hợp. Ngược lại, yêu cầu độ tin cậy phải rất cao thì chúng ta phải sử dụng các phương pháp tinh vi để kiểm thử một cách kỹ lưỡng nhất có thể. Khi chọn một phương pháp, điều quan trọng là phải đánh giá tất cả các yếu tố và chi phí liên quan.

Chi phí nền tảng kiểm thử: Mỗi đặc tả ca kiểm thử sẽ phải được cụ thể hóa thành một ca kiểm thử và được chạy đi chạy lại nhiều lần trong suốt vòng đời của sản phẩm. Mỗi lần chạy một ca kiểm thử bao gồm việc phải đưa đầu vào vào chương trình, thực hiện chương trình, và so sánh kết quả đầu ra thực tế với đầu ra mong đợi. Nếu chúng ta tự động hóa được tất cả các công việc này thì phương pháp tổ hợp sinh ra tất cả các tổ hợp có thể áp dụng. Tuy nhiên nếu một số bước phải thực hiện bằng tay, hoặc chúng ta phải viết mã cho từng đặc tả kiểm thử này thì có lẽ chúng ta phải chọn phương pháp khác để khả thi về mặt thời gian và công sức.

Tóm lại kiểm thử hàm cũng giống nhiều hoạt động kỹ thuật khác đòi hỏi phải phân tích kỹ lưỡng các ưu nhược điểm để chọn phương pháp cân bằng phù hợp với các khó khăn và các vấn đề không lường hết được mà phải là người thiết kế có kinh nghiệm mới có cảm giác tốt được. Kiểm thử hàm không phải là việc tập để chọn một phương pháp tối ưu mà là một loạt các hoạt động để tìm ra sự tổ hợp của một số mô hình và kỹ thuật để tạo ra một tập các ca kiểm thử thỏa mãn chi phí và ràng buộc chất lượng đặt ra. Sự cân bằng này cũng có thể mở rộng ra ngoài thiết kế kiểm

thử sang thiết kế phần mềm để thuận lợi cho việc kiểm thử. Thiết kế phần mềm thích hợp sẽ không chỉ giúp quá trình phát triển được tốt hơn mà còn giúp việc kiểm thử cũng thuận lợi để giúp tiết kiệm chi phí tổng thể của dự án phần mềm.

5.2 Kiểm thử bảng quyết định

Kỹ thuật kiểm thử lớp tương đương và kiểm thử giá trị biên thích hợp cho các hàm có các biến đầu vào không có quan hệ ràng buộc với nhau. Kỹ thuật kiểm thử dựa trên bảng quyết định chúng ta xem xét trong chương này sẽ phù hợp cho các hàm có các hành vi khác nhau dựa trên tính chất của bộ giá trị của đầu vào.

Kiểm thử dựa trên bảng quyết định là phương pháp chính xác nhất trong các kỹ thuật kiểm thử chức năng. Bảng quyết định là phương pháp hiệu quả để mô tả các sự kiện, hành vi sẽ xảy ra khi một số điều kiện thỏa mãn.

Cấu trúc Bảng quyết định:

Cấu trúc bảng quyết định chia thành bốn phần chính như trong Bảng 5.1:

- Các biểu thức điều kiện C1, C2, C3.
- Giá trị điều kiện T, F, –.
- Các hành động A1, A2, A3, A4.
- Giá trị hành động, có (xảy ra) hay không, X là có.

Bảng 5.1: Ví dụ bảng quyết định

Điều kiện	Các trường hợp (qui tắc)			
	TH ₁	TH ₂	TH ₃	TH ₄
đk1 (C ₁)	T	T	F	F
đk2 (C ₂)	T	F	T	F
Hành động của hệ thống				
A ₁	T (Có)	F (không)	T	F
A ₂	T	T	F	F

Khi lập bảng quyết định ta thường tìm các điều kiện có thể xảy ra, để xét các tổ hợp của chúng mà từ đó chúng ta sẽ xác định được các ca kiểm thử tương ứng cho các điều kiện được thỏa mãn. Các hành động xảy ra chính là kết quả mong đợi của ca kiểm thử đó.

Bảng quyết định với các giá trị điều kiện chỉ là T, F, và – được gọi là *bảng quyết định logic*. Chúng ta có thể mở rộng các giá trị này bằng các tập giá trị khác, ví dụ 1, 2, 3, 4, khi đó chúng ta có *bảng quyết định tổng quát*.

Kỹ thuật thực hiện: Để xác định các ca kiểm thử dựa trên bảng quyết định, ta dịch các điều kiện thành các đầu vào và các hành động thành các đầu ra. Đôi khi các điều kiện sẽ xác định các lớp tương đương của đầu vào và các hành động tương ứng với các mô-đun xử lý chức năng đang kiểm thử.

Do đó mỗi cột tương ứng với một ca kiểm thử. Vì tất cả các cột bao phủ toàn bộ các tổ hợp đầu vào nên chúng ta có một bộ kiểm thử đầy đủ.

Trên thực tế không phải tổ hợp đầu vào nào cũng là hợp lệ, do đó khi sử dụng bảng quyết định người ta thường bổ sung thêm một giá trị đặc biệt ‘–’ để đánh dấu các điều kiện không thể cùng xảy ra này. Các giá trị – (không quan tâm) có thể hiểu là luôn sai, không hợp lệ. Nếu các điều kiện chỉ là T và F ta có 2^n cột qui tắc.

Bảng 5.2 là một ví dụ đơn giản về một bảng quyết định để khắc phục sự cố máy in. Khi máy in có sự cố, chúng ta sẽ xem xét tình trạng dựa trên các điều kiện trong bảng là đúng hay sai, từ đó xác định được cột duy nhất có các điều kiện thỏa mãn, và thực hiện các hành động khắc phục sự cố tương ứng.

Bảng 5.2: Bảng quyết định để khắc phục sự cố máy in

Điều kiện	Máy in không in	Y	Y	Y	Y	N	N	N	N
	Đèn đỏ nhấp nháy	Y	Y	N	N	Y	Y	N	N
	Không nhận ra máy in	Y	N	Y	N	Y	N	Y	N
Hành động	Kiểm tra dây nguồn				X				
	Kiểm tra cáp máy in	X			X				
	Kiểm tra phần mềm in	X			X			X	
	Kiểm tra mực in	X	X			X	X		
	Kiểm tra kẹt giấy		X		X				

Chú ý là ở đây thứ tự các điều kiện và thứ tự thực hiện hành động không quan trọng, nên chúng ta có thể đổi vị trí các hàng. Với các hành động cũng vậy, tuy nhiên tùy trường hợp chúng ta có thể làm mịn hơn bằng việc đánh số thứ tự hành động xảy ra thay cho dấu X để chỉ ra hành động nào cần làm trước. Với bảng quyết định tổng quát, các giá trị của điều kiện không chỉ nhận giá trị đúng (T) hoặc sai (F), khi đó ta cần tăng số cột để bao hết các tổ hợp có thể của các điều kiện.

Kinh nghiệm áp dụng:

So với các kỹ thuật kiểm thử khác, kiểm thử bằng bảng quyết định tốt hơn với một số bài toán (như NextDate) nhưng cũng kém hơn với một số bài toán (như Commission). Những bài toán phù hợp với bảng quyết định khi chương trình có nhiều lệnh rẽ nhánh (như Triangle) và các biến đầu vào có quan hệ với nhau (như NextDate).

1. Nên dùng kỹ thuật bảng quyết định cho các ứng dụng có một trong các tính chất sau:

- Chương trình có nhiều lệnh rẽ nhánh – nhiều khối If-Then-Else
- Các biến đầu vào có quan hệ với nhau
- Có các tính toán giữa các biến đầu vào
- Có quan hệ nhân quả giữa đầu vào và đầu ra
- Có độ phức tạp cao Cyclomatic cao

2. Bảng quyết định không dễ áp dụng cho các bài toán lớn (với n điều kiện có 2^n quy tắc). Nên dùng dạng mở rộng và sử dụng đại số để đơn giản hóa bảng.

Có thể cần một số lần thử và rút kinh nghiệm để dần dần lập được bảng tối ưu.

5.3 Kiểm thử tổ hợp

Thông thường lỗi phần mềm do một điều kiện nào đó làm phát sinh, không liên quan gì đến các điều kiện khác. Ví dụ khi nhập ngày sinh của một học sinh lớp bốn là 19/8/2013 vào danh sách lớp năm học 2013-2014 thì thông báo lỗi cần xuất hiện vì học sinh này chưa được một tuổi. Tuy nhiên cũng có những lỗi phần mềm chỉ xuất hiện do kết hợp của một số điều kiện. Để kiểm tra kỹ một phần mềm chúng ta phải kiểm tra tất cả các tổ hợp của các điều kiện có thể. Nhưng thông thường số tổ hợp của tất cả các điều kiện này sẽ bùng nổ rất nhanh và sẽ rất tốn công để kiểm tra tất cả chúng. Ví dụ giả sử hàm $y = f(x_1, x_2)$ với x_1 và x_2 chỉ nhận giá trị nguyên từ 1 đến 5. Chúng ta có $5 * 5 = 25$ tổ hợp phải kiểm thử là $(1, 1), (1, 2), \dots, (5, 5)$.

Tổng quát hóa với hàm $Y = P(X)$ trong đó $X = (x_1, \dots, x_n)$ và để đơn giản ta giả sử với mỗi biến x_i , $1 \leq i \leq n$ chỉ có k_i giá trị đáng chú ý. Nếu tính tất cả các tổ hợp

×

của các giá trị đáng chú ý này chúng ta có $k_1 \dots k_n$ ca kiểm thử.

Bằng cách không lấy hết tất cả các tổ hợp có thể mà chúng ta chỉ yêu cầu mọi tổ hợp của n biến bất kỳ đều xuất hiện trong một ca kiểm thử nào đó thì chúng ta có thể giảm đáng kể số ca kiểm thử so với tất cả các tổ hợp có thể. Kết quả nghiên cứu thực tế cho thấy tổ hợp đôi một đã có khả năng phát hiện khoảng 80% lỗi. Kiểm thử tổ hợp bốn đến sáu biến có thể phát hiện được 100% lỗi [KWG04]. Chúng ta sẽ xem xét một dạng phổ biến nhất với $n = 2$ gọi là kiểm thử đôi một.

5.3.1 Kiểm thử đôi một

Kiểm thử đôi một (pairwise testing) là một trường hợp đặc biệt của kiểm thử tất cả các tổ hợp. Kiểm thử đôi một chỉ yêu cầu mỗi cặp giá trị của (x_i, x_j) xuất hiện trong một ca kiểm thử nào đó. Một ca kiểm thử thường có nhiều cặp giá trị này với các i, j khác nhau nên dễ thấy số lượng ca kiểm thử sẽ giảm đáng kể so với tổ hợp tất cả các ca kiểm thử.

Với kiểm thử đôi một, mỗi cặp giá trị của hai biến bất kỳ cần xuất hiện trong ít nhất một ca kiểm thử. Bảng 5.3 là bộ kiểm thử đôi một, tuy chỉ có bốn ca kiểm thử, nhưng chúng chứa tất cả các cặp giá trị có thể của các biến.

Bảng 5.3: Các ca kiểm thử đôi một cho hàm g

TT	x_1	x_2	x_3
TC1	<i>True</i>	0	a
TC2	<i>True</i>	5	b
TC3	<i>False</i>	0	b
TC4	<i>False</i>	5	a

Bảng 5.4: Mảng trực giao $L_4(2^3)$

TT	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2

$$\begin{array}{c|ccc} 4 & 2 & 2 & 1 \\ \hline \end{array}$$

5.3.2 Ma trận trực giao

Chúng ta hay xem xét mảng hai chiều trong Bảng 5.4. Bảng này có hai tính chất thú vị. Tất cả các cặp (1,1), (1,2), (2,1), and (2,2) đều xuất hiện trong hai cột bất kỳ. Tuy nhiên không phải tất cả các tổ hợp của 1 và 2 đều có trong bảng. Ví dụ (2, 2, 2) là tổ hợp hợp lệ nhưng nó không nằm trong bảng. Chỉ bốn trong tám tổ hợp nằm trong bảng. Trong bảng chỉ có 4 tổ hợp trên tổng số 8 tổ hợp có thể. Đây là một ví dụ về ma trận trực giao $L_4(2^3)$. Chỉ số 4 là số hàng của mảng, 3 là số cột của mảng, và 2 là số giá trị lớn nhất một phần tử của mảng có thể nhận.

❖ Kinh nghiệm áp dụng

Đến đây chúng ta đã hiểu về khái niệm ma trận trực giao và ứng dụng của nó trong việc sinh các ca kiểm thử cặp đôi. Sau đây là một qui trình thực hiện việc ứng dụng ma trận trực giao để sinh các ca kiểm thử theo phương pháp kiểm thử cặp đôi.

Bước 1. Xác định tất cả các biến đầu vào, vì chúng sẽ tương ứng với các cột của ma trận trực giao.

Bước 2. Xác định số lượng giá trị của mỗi biến có thể nhận để xác định được số lượng lớn nhất vì các giá trị của ma trận sẽ nằm trong khoảng này.

Bước 3. Tìm ma trận trực giao thích hợp với số lượng hàng nhỏ nhất với hai số cột và giá trị của các phần tử ma trận đã xác định ở Bước 1 và Bước 2.

Bước 4. Ánh xạ các biến với các cột của ma trận và giá trị của biến với các phần tử của ma trận.

Bước 5. Kiểm tra còn phần tử nào trong ma trận chưa được ánh xạ không. Chọn các giá trị hợp lệ tùy ý cho những phần tử này.

Bước 6. Chuyển các hàng của ma trận thành các ca kiểm thử

Bài tập cuối bài:

1. Cấu trúc của bảng quyết định?
2. Các bước thực hiện kiểm thử bằng bảng quyết định?
3. Các bước thực hiện kiểm thử tổ hợp?
4. Kinh nghiệm áp dụng các kỹ thuật kiểm thử hàm?
5. Các bài tập thực hành:

Bài 1:

Áp dụng kỹ thuật kiểm thử dựa trên bảng quyết định thực hiện kiểm thử chức năng Xác định giá vé máy bay.

Mô tả: Giá vé máy bay được tính như sau:

Hạng vé: Phổ thông, Thương gia.

Độ tuổi: Dưới 2, từ 2-60, trên 60.

Thời gian đặt vé trước ngày bay: Dưới 7 ngày, từ 7-30 ngày, trên 30 ngày.

Thời điểm bay: Giờ cao điểm, giờ bình thường.

Yêu cầu:

1. Xây dựng bảng quyết định
2. Viết kịch bản kiểm thử

Bài 2:

Áp dụng kỹ thuật kiểm thử dựa trên bảng quyết định thực hiện kiểm thử chức năng Phân loại sản phẩm.

Mô tả: Sản phẩm được phân loại dựa trên:

Loại sản phẩm: Điện tử, Gia dụng.

Mức giá: Dưới 1 triệu, từ 1-5 triệu, trên 5 triệu.

Thương hiệu: Cao cấp, Bình dân.

Thời gian bảo hành: Dưới 6 tháng, 6-12 tháng, trên 12 tháng.

Yêu cầu:

1. Xây dựng bảng quyết định
2. Viết kịch bản kiểm thử

Bài 3:

Áp dụng kỹ thuật kiểm thử dựa trên bảng quyết định thực hiện kiểm thử chức năng Tuyển dụng.

Mô tả: Quyết định tuyển dụng ứng viên dựa trên:

Bằng cấp: Đại học, Thạc sĩ, Tiến sĩ.

Kinh nghiệm: Dưới 3 năm, trên 3 năm.

Kỹ năng ngoại ngữ: Cơ bản, Tốt.

Độ tuổi: Dưới 25, 25-35, trên 35.

Yêu cầu:

1. Xây dựng bảng quyết định
2. Viết kịch bản kiểm thử

Bài 4:

Áp dụng kỹ thuật kiểm thử dựa trên bảng quyết định thực hiện kiểm thử chức năng Phân loại khách hàng.

Mô tả: Khách hàng được phân loại dựa trên:

Khu vực: Thành thị, nông thôn.

Mức chi tiêu hàng tháng: Dưới 5 triệu, từ 5-20 triệu, trên 20 triệu.

Thời gian gắn bó với công ty: Dưới 1 năm, 1-3 năm, trên 3 năm.

Đăng ký thành viên: Có, Không.

Yêu cầu:

1. Xây dựng bảng quyết định
2. Viết kịch bản kiểm thử

Bài 5:

Áp dụng kỹ thuật kiểm thử dựa trên bảng quyết định thực hiện kiểm thử chức năng Tính lãi suất ngân hàng.

Mô tả: Lãi suất được tính dựa trên:

Loại hình gửi tiền: Có kỳ hạn, Không kỳ hạn.

Số tiền gửi: Dưới 50 triệu, 50-500 triệu, trên 500 triệu.

Thời gian gửi: Dưới 6 tháng, 6-12 tháng, trên 12 tháng.

Khách hàng VIP: Có, Không.

Yêu cầu:

1. Xây dựng bảng quyết định
2. Viết kịch bản kiểm thử